

claude-windows / b13673cc-f087-43b1-9527-35dc0b67b301

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\b13673cc-f087-43b1-9527-35dc0b67b301\subagents\workflows\wf_61401384-db8\agent-a1f4ce01de29464b0.jsonl`  
- SHA-256: `e8ab2200f73a2f8afedb1fe6b1bd6202fe474694a97a5699c70f2fd7c7bf911d`  
- Source modified: `2026-06-06T06:48:28+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_61401384-db8`  
- session_id: `b13673cc-f087-43b1-9527-35dc0b67b301`
```

Transcript

1. user (2026-06-06T06:45:13.233Z)

You are auditing how the badminton-highlight-indexer consumes ground-truth rally annotations, to determine whether a third-party vendor (Roora) delivery can be integrated.

KEY PATHS (all local, absolute, read with the Read/Bash/Grep tools):

```
- Roora delivery root: "C:\Users\avidu\Projects\Annotation Setup\6 June 2026"  
  - "badminton\" = PASCAL VOC export (Annotations\frame_XXXXXX.xml per sampled frame, ImageSets\, labelmap.txt)  
  - "badminton (1)\annotations\instances_Test.json" = COCO export  
  - "badminton (2)\annotations.xml" = CVAT-for-video native export (has <meta><job> with frame_filter step, stop_frame, labels)  
- Roora spec PDFs the user sent the vendor:  
  - "C:\Users\avidu\Projects\Annotation Setup\Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf"  
  - "C:\Users\avidu\Projects\Annotation Setup\Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf"  
- The three sample videos given: "G:\My Drive\Sharing\Annotation Videos\" (Video 1 - Badminton.MP4, Video 2 - Pickleball.mp4, Video 3 - Tabletennis.mp4)  
- Indexer repo: C:\Users\avidu\Projects\badminton-highlight-indexer  
- Collector repo: C:\Users\avidu\Projects\sports-data-collector  
- python available as 'python' (3.13); ffmpeg available.
```

FACTS ALREADY ESTABLISHED (verify, don't just trust; correct me if any is wrong):

```
- ffmpeg on the on-disk videos: Badminton = 5312x2988, 29.97fps, 100.7s, 3019 frames.  
Pickleball = 3840x2160, 59.94fps, ~97s. Tabletennis = 3840x2160, 59.94fps, ~58s.  
- CVAT job meta (badminton (2)/annotations.xml): stop_frame=21270, frame_filter step=30, size=710 sampled frames, annotation canvas 1920x1080, mode=interpolation, label="rally".  
- All three folders appear to be the SAME badminton job exported in 3 formats (COCO bbox [1134.1,434.73,152.19,270.73] == VOC rally-2 box xmin1134.1/ymin434.73/xmax1286.29/ymax705.46). Pickleball & table-tennis are NOT in the delivery.  
- The rally fields delivered (as CVAT object attributes, attached to a per-player bounding box): rally_number, start_time, end_time, shots_count, ending_reason, score_before (A-B), score_after (A-B), notes, rotation.  
- VOC has 24 distinct rally_numbers; 281/710 frames contain >=1 rally object; COCO has 318 annotations over 710 images.  
- HYPOTHESIS to verify: start_time/end_time attributes == source_frame/900 (e.g. rally 24 frame 19920->20160 gives 22.13->22.4; 19920/900=22.13). i.e. attribute times are off by the sampling-step factor and the TRUE temporal signal is the per-object FRAME RANGE, not the start_time/end_time attributes.  
- HYPOTHESIS: the annotated badminton source (21270 frames, 1080p) is NOT the on-disk Video 1 - Badminton.MP4 (3019 frames, 5312x2988).  
- ending_reason values used: forced_error, unforced_error, service_fault, let, other, winner.
```

TASK: Read the indexer eval + annotations code and the collector exporter to define the EXACT integration contract:

- Indexer: backend/eval/metrics.py, backend/eval/regression.py, backend/eval/batch.py, backend/eval/annotations.py, backend/annotations/file_provider.py, backend/annotations/automated_provider.py
- Collector: src/cli/curate.py (export_eval_set), src/core/db.py (get_segment_intervals, SEGMENT_TABLES)
- Also skim indexer docs/GOLDEN_SET_IMPLEMENTATION.md and docs/GOLDEN_SET_VENDORING.md for the intended provider contract.

Determine: the exact GT CSV/interval schema the pipeline expects (columns, units ? seconds? ? and [start,end) semantics), how the providers/metrics load intervals, and exactly what an adapter would have to emit to turn ONE Roora export into valid GT. Pick which of the 3 Roora formats (VOC/COCO/CVAT-video) is easiest to adapt and sketch the adapter. Estimate effort and list blockers. Cite file:line for every claim.

2. assistant (2026-06-06T06:45:16.074Z)

I'll audit the integration contract. Let me start by reading the indexer eval and annotations code in parallel.

3. user (2026-06-06T06:45:17.021Z)

```
1      """Pure temporal interval-matching metrics for rally evaluation.
2
3      No I/O, no DB, no global state ? every function here is deterministic and unit
4      tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5      start < end. We score a list of *predicted* rally intervals against a list of
6      *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7      """
8
9      from dataclasses import dataclass, field
10     from typing import List, Tuple, Optional
11
12     Interval = Tuple[float, float]
13
14
15     def interval_iou(a: Interval, b: Interval) -> float:
16         """Temporal intersection-over-union of two intervals.
17
18         Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19         overlap / union, both measured as durations on the timeline.
20         """
21         a_start, a_end = a
22         b_start, b_end = b
23         if a_end <= a_start or b_end <= b_start:
24             return 0.0
25         inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26         if inter <= 0.0:
27             return 0.0
28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass
41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
```

```

44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                       iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52         Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53         are claimed highest-IoU first, and each prediction/GT can be used once. This
54         keeps a single prediction from being credited for two ground-truth rallies
55         (and vice versa).
56         """
57         candidates = []
58         for pi, p in enumerate(preds):
59             for gi, g in enumerate(gts):
60                 iou = interval_iou(p, g)
61                 if iou >= iou_threshold:
62                     candidates.append((iou, pi, gi))
63         # Highest IoU first; ties broken by indices for determinism.
64         candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
65
66         used_pred, used_gt = set(), set()
67         matches: List[Match] = []
68         for iou, pi, gi in candidates:
69             if pi in used_pred or gi in used_gt:
70                 continue
71             used_pred.add(pi)
72             used_gt.add(gi)
73             matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
74
75         unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
76         unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
77         return MatchResult(matches=matches,
78                           unmatched_pred_indices=unmatched_pred,
79                           unmatched_gt_indices=unmatched_gt)
80
81
82     @dataclass
83     class Score:
84         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
85         iou_threshold: float
86         num_pred: int
87         num_gt: int
88         true_positives: int
89         false_positives: int
90         false_negatives: int
91         precision: float
92         recall: float
93         f1: float
94         mean_iou: float # over matched pairs (0.0 if none)
95         mean_start_error: float # mean |pred_start - gt_start| over matches, seconds
96         mean_end_error: float # mean |pred_end - gt_end| over matches, seconds
97
98         def as_dict(self) -> dict:
99             return self.__dict__.copy()
100
101
102     def score(preds: List[Interval], gts: List[Interval],
103             iou_threshold: float = 0.5,
104             match_result: Optional[MatchResult] = None) -> Score:
105         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107         Pass a precomputed `match_result` to avoid re-matching (e.g. when also

```

```

108         extracting the failure lists); otherwise matching is done here.
109         """
110         mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
111
112         tp = len(mr.matches)
113         fp = len(mr.unmatched_pred_indices)
114         fn = len(mr.unmatched_gt_indices)
115
116         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
117         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
118         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
119
120         if mr.matches:
121             mean_iou = sum(m.iou for m in mr.matches) / tp
122             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
123             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
124         else:
125             mean_iou = mean_start_err = mean_end_err = 0.0
126
127         return Score(
128             iou_threshold=iou_threshold,
129             num_pred=len(preds), num_gt=len(gts),
130             true_positives=tp, false_positives=fp, false_negatives=fn,
131             precision=precision, recall=recall, f1=f1,
132             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
133         )
134
135
136 def pr_curve(preds: List[Interval], gts: List[Interval],
137             thresholds: Optional[List[float]] = None) -> List[Score]:
138     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
139
140     Lets you see strict-vs-lenient behaviour rather than a single operating point.
141     """
142     if thresholds is None:
143         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144     return [score(preds, gts, iou_threshold=t) for t in thresholds]
145

```

4. user (2026-06-06T06:45:17.491Z)

```

1     """Load hand-annotated ground-truth rally timestamps from CSV.
2
3     Annotation file format (one file per video):
4
5         start,end
6         00:01:12,00:01:39
7         102.5,131.0
8
9     - A header row `start,end` is optional (auto-detected and skipped).
10    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12    - Blank lines and `#` comment lines are ignored.
13    """
14
15    import csv
16    import logging
17    from typing import List, Tuple
18
19    logger = logging.getLogger(__name__)
20

```

```

21     Interval = Tuple[float, float]
22
23
24 def parse_timestamp(value: str) -> float:
25     """Parse a timestamp string to float seconds.
26
27     Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28     Raises ValueError on anything unparseable so bad annotations fail loudly.
29     """
30     value = value.strip()
31     if not value:
32         raise ValueError("empty timestamp")
33     if ":" in value:
34         parts = value.split(":")
35         if len(parts) > 3:
36             raise ValueError(f"too many ':' segments in timestamp '{value}'")
37         total = 0.0
38         for part in parts:
39             total = total * 60 + float(part)
40         return total
41     return float(value)
42
43
44 def load_annotations(csv_path: str) -> List[Interval]:
45     """Load and validate ground-truth rally intervals from a CSV file.
46
47     Returns intervals sorted by start time. Raises ValueError on malformed rows
48     (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49     than silently skewing scores.
50     """
51     intervals: List[Interval] = []
52     with open(csv_path, newline="") as f:
53         reader = csv.reader(f)
54         for lineno, row in enumerate(reader, start=1):
55             if not row:
56                 continue
57             first = row[0].strip()
58             if not first or first.startswith("#"):
59                 continue
60             # Skip an optional header row.
61             if first.lower() in ("start", "start_time"):
62                 continue
63             if len(row) < 2:
64                 raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65             try:
66                 start = parse_timestamp(row[0])
67                 end = parse_timestamp(row[1])
68             except ValueError as e:
69                 raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70             if start >= end:
71                 raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72             intervals.append((start, end))
73
74     intervals.sort(key=lambda iv: iv[0])
75     logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76     return intervals
77
78
79 def write_scaffold(csv_path: str) -> None:
80     """Write an empty annotation file with just the header, for hand-filling."""
81     with open(csv_path, "w", newline="") as f:
82         writer = csv.writer(f)
83         writer.writerow(["start", "end"])
84

```

5. user (2026-06-06T06:45:17.891Z)

```
1      """FileProvider ? ground-truth rally labels from pre-made CSV files.
2
3      This formalizes today's manual / ShuttleSet annotation flow behind the
4      `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
5      exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
6      job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.
7
8      It's the synchronous, zero-cost provider used as the default golden-set source and
9      as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
10     """
11     import os
12     from typing import List, Optional
13
14     from backend.annotations.base import (
15         AnnotationProvider,
16         annotation_registry,
17         Interval,
18         DONE,
19         FAILED,
20     )
21     from backend.eval.annotations import load_annotations
22
23
24     class FileProvider(AnnotationProvider):
25         def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
26             """
27             Args:
28                 csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
29                         `
```

```
64
65
66     annotation_registry.register(FileProvider)
67
```

6. assistant (2026-06-06T06:45:20.712Z)

Let me read the remaining indexer eval files and the automated provider.

7. user (2026-06-06T06:45:21.677Z)

```
1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7         ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8         predictions  ? the production pipeline's stored DB rows      [backend.eval.harness]
9         comparison   ? interval matching + P/R/F1                    [backend.eval.metrics]
10        baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import logging
20    from dataclasses import dataclass, asdict
21    from typing import Dict, List, Optional
22
23    from backend.database import Database
24    from backend.eval import metrics
25    from backend.eval.harness import get_predictions
26    from backend.annotations import annotation_registry
27
28    logger = logging.getLogger(__name__)
29
30    DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31    DEFAULT_TOLERANCE = 0.05
32
33
34    @dataclass
35    class RegressionResult:
36        video_id: str
37        stage: str
38        iou_threshold: float
39        precision: float
40        recall: float
41        f1: float
42        # Baseline values compared against (None when no baseline existed yet).
43        baseline_precision: Optional[float]
44        baseline_recall: Optional[float]
45        tolerance: float
46        regressed: bool
47        reasons: List[str]
48
49        def as_dict(self) -> dict:
50            return asdict(self)
51
52
53    # ----- baseline store
54
```

```

55     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
56         """Load the per-video baseline snapshot file (returns {} if absent)."""
57         if not os.path.isfile(path):
58             return {}
59         with open(path) as f:
60             return json.load(f)
61
62
63     def _baseline_key(video_id: str, stage: str) -> str:
64         return f"{video_id}::{stage}"
65
66
67     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
68                     f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
69         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
70
71         Use this to (re)baseline after a *legitimate* production improvement, so old
72         baselines don't raise false regressions. Stored as plain JSON for easy diffing.
73         """
74         baselines = load_baselines(path)
75         baselines[_baseline_key(video_id, stage)] = {
76             "video_id": video_id, "stage": stage,
77             "precision": precision, "recall": recall, "f1": f1,
78         }
79         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
80         with open(path, "w") as f:
81             json.dump(baselines, f, indent=2, sort_keys=True)
82         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
83                     _baseline_key(video_id, stage), precision, recall)
84
85     # ----- the runner
86
87     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
88               stage: str = "final", iou_threshold: float = 0.5,
89               detector: Optional[str] = None,
90               tolerance: float = DEFAULT_TOLERANCE,
91               baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
92         """Score stored predictions for one video against ground truth and compare to
93         baseline.
94
95         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
96         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
97         precision OR recall falls more than `tolerance` below the snapshotted baseline.
98         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
99         caller can choose to snapshot the current numbers via `save_baseline`.
100
101         """
102         preds = get_predictions(db, video_id, stage, detector=detector)
103         s = metrics.score(preds, ground_truth, iou_threshold)
104
105         baselines = load_baselines(baseline_path)
106         b = baselines.get(_baseline_key(video_id, stage))
107
108         reasons: List[str] = []
109         regressed = False
110         base_p = base_r = None
111         if b is not None:
112             base_p, base_r = b["precision"], b["recall"]
113             if s.precision < base_p - tolerance:
114                 regressed = True
115                 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
116 {tolerance}")
117             if s.recall < base_r - tolerance:
118                 regressed = True
119                 reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -

```

```

{tolerance}")
117     else:
118         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
119
120     return RegressionResult(
121         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
122         precision=s.precision, recall=s.recall, f1=s.f1,
123         baseline_precision=base_p, baseline_recall=base_r,
124         tolerance=tolerance, regressed=regressed, reasons=reasons,
125     )
126
127
128 def regress_with_provider(db: Database, video_id: str, video_ref: str,
129                          tier: str = "file", stage: str = "final",
130                          iou_threshold: float = 0.5,
131                          detector: Optional[str] = None,
132                          tolerance: float = DEFAULT_TOLERANCE,
133                          baseline_path: str = DEFAULT_BASELINE_PATH,
134                          guideline: Optional[str] = None,
135                          sport: str = "badminton",
136                          provider_kwargs: Optional[dict] = None) -> RegressionResult:
137     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
138
139     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
140     once those providers land). `video_ref` is whatever that provider expects (a CSV
141     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
142     predictions in the DB (the file MD5).
143     """
144     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
145     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
146     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
147                  detector=detector, tolerance=tolerance, baseline_path=baseline_path)
148
149
150 def format_result(r: RegressionResult) -> str:
151     verdict = "REGRESSED" if r.regressed else "ok"
152     head = (f"[{verdict}] {r.video_id} ({r.stage}) "
153            f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154     if r.baseline_precision is not None:
155         head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156     return head + " ".join(f"\n    - {why}" for why in r.reasons)
157

```

8. user (2026-06-06T06:45:22.141Z)

```

1     """Batch evaluation across a collector export manifest.
2
3     Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4     sports-data-collector's `curate export`, (optionally) process each video through
5     a segmenter so detections land in the indexer DB, then score every video's
6     detected rallies against its ground-truth CSV and aggregate the results.
7
8     This module holds the *pure* pieces (path resolution, stage selection, metric
9     aggregation) so they can be unit-tested without touching video files, the GPU,
10    or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11    """
12
13    import os
14    from typing import Dict, List, Optional
15
16
17    # Segmenters that only ever populate the `final` play_segments table (no raw
18    # candidate stage), so scoring `candidates` for them would be meaninglessly empty.

```

```

19 _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22 def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23     """Resolve a manifest `local_path` to an absolute path.
24
25     The collector stores paths like ``./data/videos/shuttleset_1.mp4`` relative
26     to its own repo root and with Windows separators. Normalise separators, strip
27     a leading ``./``, and resolve relative paths against `videos_root` (the
28     collector root). Returns None for a missing/empty path.
29     """
30     if not local_path:
31         return None
32     p = local_path.replace("\\", "/")
33     if p.startswith("./"):
34         p = p[2:]
35     if os.path.isabs(p):
36         return os.path.normpath(p)
37     return os.path.normpath(os.path.join(videos_root, p))
38
39
40 def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
41     """Pick which prediction stages to score.
42
43     ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
44     but only ``final`` for segmenters that don't emit candidates. An explicit
45     request ('candidates' | 'final' | 'both') overrides the heuristic.
46     """
47     if requested == "both":
48         return ["candidates", "final"]
49     if requested in ("candidates", "final"):
50         return [requested]
51     # auto
52     if segmenter in _FINAL_ONLY_SEGMENTERS:
53         return ["final"]
54     return ["candidates", "final"]
55
56
57 def _safe_div(num: float, den: float) -> float:
58     return num / den if den else 0.0
59
60
61 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
62     """Aggregate per-video stage scores into corpus-level metrics.
63
64     `per_video` is a list of dicts shaped like `report_to_dict` output (each has
65     `stages[stage]["score"]` with tp/fp/fn/mean_iou/...). For each stage we
66     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
67     a TP-weighted mean IoU, plus the macro mean of per-video F1.
68     """
69     out: Dict[str, dict] = {}
70     for stage in stages:
71         tp = fp = fn = 0
72         iou_weighted = 0.0
73         f1s: List[float] = []
74         n_videos = 0
75         for rep in per_video:
76             sr = rep.get("stages", {}).get(stage)
77             if not sr:
78                 continue
79             s = sr["score"]
80             tp += s["true_positives"]
81             fp += s["false_positives"]
82             fn += s["false_negatives"]
83             iou_weighted += s["mean_iou"] * s["true_positives"]

```

```

84         f1s.append(s["f1"])
85         n_videos += 1
86         precision = _safe_div(tp, tp + fp)
87         recall = _safe_div(tp, tp + fn)
88         out[stage] = {
89             "videos": n_videos,
90             "tp": tp, "fp": fp, "fn": fn,
91             "micro_precision": precision,
92             "micro_recall": recall,
93             "micro_f1": _safe_div(2 * precision * recall, precision + recall),
94             "macro_f1": _safe_div(sum(f1s), len(f1s)),
95             "mean_iou": _safe_div(iou_weighted, tp),
96         }
97     return out
98
99
100 def format_aggregate(agg: Dict[str, dict]) -> str:
101     """Human-readable corpus-level summary table."""
102     lines = [
103         "=" * 78,
104         " CORPUS AGGREGATE",
105         "=" * 78,
106         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
107         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
108         "-" * 78,
109     ]
110     for stage, a in agg.items():
111         lines.append(
112             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
113             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
114             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
115         )
116     lines.append("=" * 78)
117     return "\n".join(lines)
118

```

9. user (2026-06-06T06:45:22.684Z)

```

1     """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).
2
3     The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
4     video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
5     between authoritative human-vendor runs (the slow tier).
6
7     This slice ships the **interface only**, deliberately model-agnostic: an
8     `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
9     a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
10    tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
11    step so the lineages differ (see the oracle rule below).
12
13    ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
14    production uses. If production becomes the offline ActionFormer segmenter and the
15    oracle is also ActionFormer, they share blind spots and the regression test is
16    blind ? green while both are wrong. `lineage` records the oracle's model family so
17    callers can guard against it matching production (`warn_if_same_lineage`).
18    """
19    import logging
20    from typing import Callable, Dict, List, Optional
21
22    from backend.annotations.base import (
23        AnnotationProvider,
24        annotation_registry,
25        Interval,
26        DONE,
27        FAILED,

```

```

28     )
29
30     logger = logging.getLogger(__name__)
31
32     # A labeler turns (video_ref, guideline, sport) into rally intervals.
33     Labeler = Callable[[str, Optional[str], str], List[Interval]]
34
35
36     def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
37         """Default labeler: refuses to run until a real model is injected.
38
39         Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
40         while making accidental use of an unconfigured oracle fail loudly rather than
41         silently returning no rallies.
42         """
43         raise NotImplementedError(
44             "AutomatedProvider has no labeler configured. Inject one via "
45             "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>')."
46             "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
(GS-4)."
47         )
48
49
50     def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
51         """Warn (and return True) if the oracle shares production's model lineage.
52
53         Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
54         trusted to catch production's failures. Callers wiring CI should pass both.
55         """
56         if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
57             logger.warning(
58                 "Oracle lineage '%s' matches production's ? the regression test may be "
59                 "blind to shared failure modes. Use a different model lineage.",
60                 oracle_lineage,
61             )
62             return True
63         return False
64
65
66     class AutomatedProvider(AnnotationProvider):
67         def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
68             """
69             Args:
70                 labeler: callable producing rally intervals for a video. Defaults to a
71                         stub that raises until configured.
72                 lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
73                         so callers can enforce the oracle rule via warn_if_same_lineage.
74             """
75             self._labeler = labeler or _not_configured
76             self.lineage = lineage
77             # In-process synchronous job store: job_id -> labeled intervals.
78             self._jobs: Dict[str, List[Interval]] = {}
79
80         @property
81         def name(self) -> str:
82             return "auto"
83
84         def submit(self, video_ref: str, guideline: Optional[str] = None,
85                   sport: str = "badminton") -> str:
86             # Synchronous stub: run the labeler now and stash the result. A real async
87             # oracle (e.g. a queued API job) would kick off here and resolve in poll().
88             intervals = self._labeler(video_ref, guideline, sport)
89             intervals = sorted((float(s), float(e)) for s, e in intervals)

```

```

90         job_id = video_ref
91         self._jobs[job_id] = intervals
92         return job_id
93
94     def poll(self, job_id: str) -> str:
95         return DONE if job_id in self._jobs else FAILED
96
97     def fetch(self, job_id: str) -> List[Interval]:
98         if job_id not in self._jobs:
99             raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
100         return self._jobs[job_id]
101
102
103     annotation_registry.register(AutomatedProvider)
104

```

10. assistant (2026-06-06T06:45:25.436Z)

Now let me read the base provider interface, the collector exporter, and the collector DB, plus the docs.

11. user (2026-06-06T06:45:26.399Z)

```

1     """Annotation provider registry ? pluggable sources of ground-truth rally labels.
2
3     An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
4     intervals ? the same shape that `backend.eval.annotations.load_annotations`
5     produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
6     golden-set ground truth can come from a CSV file, a human annotation vendor, or an
7     automated oracle behind one interface, so the regression flow
8     (`backend.eval.regression`) never needs to know where labels came from.
9
10    Mirrors the existing registry pattern in `backend/providers/base.py` and
11    `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
12    design rationale.
13    """
14    from abc import ABC, abstractmethod
15    from typing import Dict, List, Optional, Tuple, Type
16
17    # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
18    Interval = Tuple[float, float]
19
20    # Job lifecycle statuses returned by poll().
21    QUEUED = "queued"
22    RUNNING = "running"
23    DONE = "done"
24    FAILED = "failed"
25    STATUSES = (QUEUED, RUNNING, DONE, FAILED)
26
27
28    class AnnotationProvider(ABC):
29        """Abstract source of ground-truth rally intervals for a video.
30
31        The three-method contract (submit ? poll ? fetch) accommodates both
32        synchronous providers (a file already on disk) and asynchronous ones (a human
33        vendor that takes days). Synchronous providers can use the `annotate()`
34        convenience, which runs the full cycle in one call.
35        """
36
37        @property
38        @abstractmethod
39        def name(self) -> str:
40            """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""
41

```

```

42     @abstractmethod
43     def submit(self, video_ref: str, guideline: Optional[str] = None,
44                sport: str = "badminton") -> str:
45         """Submit a video for annotation; return an opaque job_id.
46
47         `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
48         `guideline` is the annotation spec (what counts as a rally start/end).
49         """
50
51     @abstractmethod
52     def poll(self, job_id: str) -> str:
53         """Return one of STATUSES for the given job."""
54
55     @abstractmethod
56     def fetch(self, job_id: str) -> List[Interval]:
57         """Return the completed annotations as sorted `(start, end)` intervals.
58
59         Should raise if the job is not DONE / the labels are unavailable, rather
60         than returning an empty list (which would silently skew downstream scores).
61         """
62
63     def annotate(self, video_ref: str, guideline: Optional[str] = None,
64                 sport: str = "badminton") -> List[Interval]:
65         """Convenience: submit + fetch for a provider that completes synchronously.
66
67         Async providers (e.g. a human vendor with a multi-day SLA) should be driven
68         via submit/poll/fetch explicitly instead ? calling this on one will raise
69         once the job isn't immediately DONE.
70         """
71         job_id = self.submit(video_ref, guideline=guideline, sport=sport)
72         status = self.poll(job_id)
73         if status != DONE:
74             raise RuntimeError(
75                 f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
76                 f"Use submit/poll/fetch for asynchronous providers."
77             )
78         return self.fetch(job_id)
79
80
81     class AnnotationProviderRegistry:
82         """Registry of annotation providers, keyed by each provider's `name`."""
83
84         def __init__(self):
85             self._providers: Dict[str, Type[AnnotationProvider]] = {}
86
87         def register(self, provider_cls: Type[AnnotationProvider]) ->
88         Type[AnnotationProvider]:
89             """Register a provider class under its `name`. Returns the class (decorator-
90             friendly)."""
91             try:
92                 key = provider_cls().name
93             except Exception:
94                 # Providers needing constructor args fall back to the class name.
95                 key = getattr(provider_cls, "__name__", "unknown")
96             self._providers[key] = provider_cls
97             return provider_cls
98
99         def get(self, name: str, **kwargs) -> AnnotationProvider:
100             """Instantiate the named provider, passing through any constructor kwargs."""
101             cls = self._providers.get(name)
102             if cls is None:
103                 raise KeyError(
104                     f"Unknown annotation provider '{name}'. Available:
105                     {self.list_available()}"
106                 )

```

```

104         return cls(**kwargs)
105
106     def list_available(self) -> List[str]:
107         return list(self._providers.keys())
108
109
110     annotation_registry = AnnotationProviderRegistry()
111

```

12. user (2026-06-06T06:45:27.231Z)

```

1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22    }
23
24
25    class SportsDatabase:
26        def __init__(self, db_path: str):
27            self.db_path = db_path
28            # Ensure target directory exists
29            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30            self._init_db()
31
32        @contextmanager
33        def _get_connection(self):
34            """Yield a SQLite connection and always close it.
35
36            Used as ``with self._get_connection() as conn:``. A plain
37            ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38            transaction but does NOT close the connection, which leaks handles and
39            raises ResourceWarnings. This wrapper guarantees the connection is closed
40            on exit; writers still commit explicitly inside the block.
41            """
42            conn = sqlite3.connect(self.db_path)
43            conn.row_factory = sqlite3.Row
44            # SQLite disables foreign keys per-connection by default; enable so the
45            # ON DELETE CASCADE declarations on the child tables actually fire.
46            conn.execute("PRAGMA foreign_keys = ON")
47            try:
48                yield conn
49            finally:
50                conn.close()
51
52        def _init_db(self):
53            with self._get_connection() as conn:
54                cursor = conn.cursor()

```

```

55
56 # 1. Create common videos table
57 cursor.execute("""
58 CREATE TABLE IF NOT EXISTS videos (
59     video_id TEXT PRIMARY KEY,
60     sport TEXT NOT NULL,
61     video_url TEXT,
62     local_path TEXT,
63     license_type TEXT NOT NULL,
64     license_url TEXT,
65     is_commercial INTEGER NOT NULL DEFAULT 0,
66     status TEXT NOT NULL DEFAULT 'staging',
67     match_name TEXT NOT NULL,
68     fps REAL,
69     resolution TEXT
70 )
71 """)
72
73 # 2. Create badminton_rallies table
74 cursor.execute("""
75 CREATE TABLE IF NOT EXISTS badminton_rallies (
76     video_id TEXT NOT NULL,
77     rally_number INTEGER NOT NULL,
78     start_time REAL NOT NULL,
79     end_time REAL NOT NULL,
80     score_before TEXT,
81     score_after TEXT,
82     shots_count INTEGER,
83     ending_reason TEXT,
84     PRIMARY KEY (video_id, rally_number),
85     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
86 )
87 """)
88
89 # 3. Create tennis_points table
90 cursor.execute("""
91 CREATE TABLE IF NOT EXISTS tennis_points (
92     video_id TEXT NOT NULL,
93     point_number INTEGER NOT NULL,
94     start_time REAL NOT NULL,
95     end_time REAL NOT NULL,
96     set_score TEXT,
97     game_score TEXT,
98     server TEXT,
99     ending_type TEXT,
100     PRIMARY KEY (video_id, point_number),
101     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
102 )
103 """)
104
105 # 4. Create cricket_deliveries table
106 cursor.execute("""
107 CREATE TABLE IF NOT EXISTS cricket_deliveries (
108     video_id TEXT NOT NULL,
109     over_num INTEGER NOT NULL,
110     ball_num INTEGER NOT NULL,
111     start_time REAL NOT NULL,
112     end_time REAL NOT NULL,
113     batsman TEXT,
114     bowler TEXT,
115     runs INTEGER,
116     wicket INTEGER DEFAULT 0,
117     extras INTEGER DEFAULT 0,
118     PRIMARY KEY (video_id, over_num, ball_num),
119     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE

```

```

120         )
121     """)
122
123     # 5. Create pickleball_rallies table
124     cursor.execute("""
125     CREATE TABLE IF NOT EXISTS pickleball_rallies (
126         video_id TEXT NOT NULL,
127         rally_number INTEGER NOT NULL,
128         start_time REAL NOT NULL,
129         end_time REAL NOT NULL,
130         score_before TEXT,
131         score_after TEXT,
132         shots_count INTEGER,
133         ending_reason TEXT,
134         PRIMARY KEY (video_id, rally_number),
135         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
136     )
137     """)
138
139     # 6. Create tabletennis_rallies table
140     cursor.execute("""
141     CREATE TABLE IF NOT EXISTS tabletennis_rallies (
142         video_id TEXT NOT NULL,
143         rally_number INTEGER NOT NULL,
144         start_time REAL NOT NULL,
145         end_time REAL NOT NULL,
146         score_before TEXT,
147         score_after TEXT,
148         shots_count INTEGER,
149         ending_reason TEXT,
150         PRIMARY KEY (video_id, rally_number),
151         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
152     )
153     """)
154     conn.commit()
155
156     def insert_video(self, video: VideoMetadata) -> bool:
157         """Insert a video entry, or update it in place if it already exists.
158
159         Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
160         existing row, which (with foreign keys enabled) would cascade-delete the
161         video's child annotations. UPSERT mutates the row in place and leaves
162         children untouched.
163         """
164         with self._get_connection() as conn:
165             cursor = conn.cursor()
166             cursor.execute("""
167             INSERT INTO videos (
168                 video_id, sport, video_url, local_path, license_type,
169                 license_url, is_commercial, status, match_name, fps, resolution
170             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
171             ON CONFLICT(video_id) DO UPDATE SET
172                 sport=excluded.sport,
173                 video_url=excluded.video_url,
174                 local_path=excluded.local_path,
175                 license_type=excluded.license_type,
176                 license_url=excluded.license_url,
177                 is_commercial=excluded.is_commercial,
178                 status=excluded.status,
179                 match_name=excluded.match_name,
180                 fps=excluded.fps,
181                 resolution=excluded.resolution
182             """, (
183                 video.video_id,
184                 video.sport.value,

```

```

185         video.video_url,
186         video.local_path,
187         video.license_type.value,
188         video.license_url,
189         1 if video.is_commercial else 0,
190         video.status.value,
191         video.match_name,
192         video.fps,
193         video.resolution
194     ))
195     conn.commit()
196     return True
197
198     def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
199         if not rallies:
200             return 0
201         with self._get_connection() as conn:
202             cursor = conn.cursor()
203             # First clear existing rallies for this video_id to avoid key conflicts on
updates
204             video_id = rallies[0].video_id
205             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
206
207             for rally in rallies:
208                 cursor.execute("""
209                     INSERT INTO badminton_rallies (
210                         video_id, rally_number, start_time, end_time,
211                         score_before, score_after, shots_count, ending_reason
212                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
213                 """, (
214                     rally.video_id,
215                     rally.rally_number,
216                     rally.start_time,
217                     rally.end_time,
218                     rally.score_before,
219                     rally.score_after,
220                     rally.shots_count,
221                     rally.ending_reason
222                 ))
223             conn.commit()
224             return len(rallies)
225
226     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
227         if not points:
228             return 0
229         with self._get_connection() as conn:
230             cursor = conn.cursor()
231             video_id = points[0].video_id
232             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
233
234             for pt in points:
235                 cursor.execute("""
236                     INSERT INTO tennis_points (
237                         video_id, point_number, start_time, end_time,
238                         set_score, game_score, server, ending_type
239                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
240                 """, (
241                     pt.video_id,
242                     pt.point_number,
243                     pt.start_time,
244                     pt.end_time,
245                     pt.set_score,
246                     pt.game_score,
247                     pt.server,

```

```

248         pt.ending_type
249     ))
250     conn.commit()
251     return len(points)
252
253 def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
254     if not deliveries:
255         return 0
256     with self._get_connection() as conn:
257         cursor = conn.cursor()
258         video_id = deliveries[0].video_id
259         cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
260 (video_id,))
261
262         for deliv in deliveries:
263             cursor.execute("""
264                 INSERT INTO cricket_deliveries (
265                     video_id, over_num, ball_num, start_time, end_time,
266                     batsman, bowler, runs, wicket, extras
267                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
268                 """, (
269                     deliv.video_id,
270                     deliv.over,
271                     deliv.ball,
272                     deliv.start_time,
273                     deliv.end_time,
274                     deliv.batsman,
275                     deliv.bowler,
276                     deliv.runs,
277                     1 if deliv.wicket else 0,
278                     deliv.extras
279                 ))
280             conn.commit()
281             return len(deliveries)
282
283 def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
284     if not rallies:
285         return 0
286     with self._get_connection() as conn:
287         cursor = conn.cursor()
288         video_id = rallies[0].video_id
289         cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
290 (video_id,))
291
292         for rally in rallies:
293             cursor.execute("""
294                 INSERT INTO pickleball_rallies (
295                     video_id, rally_number, start_time, end_time,
296                     score_before, score_after, shots_count, ending_reason
297                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
298                 """, (
299                     rally.video_id,
300                     rally.rally_number,
301                     rally.start_time,
302                     rally.end_time,
303                     rally.score_before,
304                     rally.score_after,
305                     rally.shots_count,
306                     rally.ending_reason
307                 ))
308             conn.commit()
309             return len(rallies)
310
311 def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
312     if not rallies:

```

```

311         return 0
312     with self._get_connection() as conn:
313         cursor = conn.cursor()
314         video_id = rallies[0].video_id
315         cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
316
317     for rally in rallies:
318         cursor.execute("""
319             INSERT INTO tabletennis_rallies (
320                 video_id, rally_number, start_time, end_time,
321                 score_before, score_after, shots_count, ending_reason
322             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
323         """, (
324             rally.video_id,
325             rally.rally_number,
326             rally.start_time,
327             rally.end_time,
328             rally.score_before,
329             rally.score_after,
330             rally.shots_count,
331             rally.ending_reason
332         ))
333     conn.commit()
334     return len(rallies)
335
336 def get_video(self, video_id: str) -> Optional[VideoMetadata]:
337     with self._get_connection() as conn:
338         cursor = conn.cursor()
339         cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
340         row = cursor.fetchone()
341         if row:
342             return VideoMetadata(
343                 video_id=row['video_id'],
344                 sport=SportType(row['sport']),
345                 video_url=row['video_url'],
346                 local_path=row['local_path'],
347                 license_type=LicenseType(row['license_type']),
348                 license_url=row['license_url'],
349                 is_commercial=bool(row['is_commercial']),
350                 status=CurationStatus(row['status']),
351                 match_name=row['match_name'],
352                 fps=row['fps'],
353                 resolution=row['resolution']
354             )
355         return None
356
357 def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
358     query = "SELECT * FROM videos WHERE status = ?"
359     params = [status.value]
360     if sport:
361         query += " AND sport = ?"
362         params.append(sport.value)
363
364     videos = []
365     with self._get_connection() as conn:
366         cursor = conn.cursor()
367         cursor.execute(query, params)
368         for row in cursor.fetchall():
369             videos.append(VideoMetadata(
370                 video_id=row['video_id'],
371                 sport=SportType(row['sport']),
372                 video_url=row['video_url'],
373                 local_path=row['local_path'],

```

```

374         license_type=LicenseType(row['license_type']),
375         license_url=row['license_url'],
376         is_commercial=bool(row['is_commercial']),
377         status=CurationStatus(row['status']),
378         match_name=row['match_name'],
379         fps=row['fps'],
380         resolution=row['resolution']
381     ))
382     return videos
383
384     def get_status_counts(self) -> List[tuple]:
385         """Return (sport, status, count) rows for the status summary.
386
387         Public read API so callers (e.g. the CLI `status` command) don't have to
388         reach into the private `_get_connection` context manager.
389         """
390         with self._get_connection() as conn:
391             cursor = conn.cursor()
392             cursor.execute(
393                 "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
394             )
395             return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]
396
397     def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
398         rallies = []
399         with self._get_connection() as conn:
400             cursor = conn.cursor()
401             cursor.execute("""
402                 SELECT * FROM badminton_rallies
403                 WHERE video_id = ?
404                 ORDER BY rally_number
405                 """, (video_id,))
406             for row in cursor.fetchall():
407                 rallies.append(BadmintonRally(
408                     video_id=row['video_id'],
409                     rally_number=row['rally_number'],
410                     start_time=row['start_time'],
411                     end_time=row['end_time'],
412                     score_before=row['score_before'],
413                     score_after=row['score_after'],
414                     shots_count=row['shots_count'],
415                     ending_reason=row['ending_reason']
416                 ))
417         return rallies
418
419     def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
420         points = []
421         with self._get_connection() as conn:
422             cursor = conn.cursor()
423             cursor.execute("""
424                 SELECT * FROM tennis_points
425                 WHERE video_id = ?
426                 ORDER BY point_number
427                 """, (video_id,))
428             for row in cursor.fetchall():
429                 points.append(TennisPoint(
430                     video_id=row['video_id'],
431                     point_number=row['point_number'],
432                     start_time=row['start_time'],
433                     end_time=row['end_time'],
434                     set_score=row['set_score'],
435                     game_score=row['game_score'],
436                     server=row['server'],
437                     ending_type=row['ending_type']

```

```

438         ))
439     return points
440
441 def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
442     deliveries = []
443     with self._get_connection() as conn:
444         cursor = conn.cursor()
445         cursor.execute("""
446         SELECT * FROM cricket_deliveries
447         WHERE video_id = ?
448         ORDER BY over_num, ball_num
449         """, (video_id,))
450         for row in cursor.fetchall():
451             deliveries.append(CricketDelivery(
452                 video_id=row['video_id'],
453                 over=row['over_num'],
454                 ball=row['ball_num'],
455                 start_time=row['start_time'],
456                 end_time=row['end_time'],
457                 batsman=row['batsman'],
458                 bowler=row['bowler'],
459                 runs=row['runs'],
460                 wicket=bool(row['wicket']),
461                 extras=row['extras']
462             ))
463     return deliveries
464
465 def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
466     rallies = []
467     with self._get_connection() as conn:
468         cursor = conn.cursor()
469         cursor.execute("""
470         SELECT * FROM pickleball_rallies
471         WHERE video_id = ?
472         ORDER BY rally_number
473         """, (video_id,))
474         for row in cursor.fetchall():
475             rallies.append(PickleballRally(
476                 video_id=row['video_id'],
477                 rally_number=row['rally_number'],
478                 start_time=row['start_time'],
479                 end_time=row['end_time'],
480                 score_before=row['score_before'],
481                 score_after=row['score_after'],
482                 shots_count=row['shots_count'],
483                 ending_reason=row['ending_reason']
484             ))
485     return rallies
486
487 def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
488     rallies = []
489     with self._get_connection() as conn:
490         cursor = conn.cursor()
491         cursor.execute("""
492         SELECT * FROM tabletennis_rallies
493         WHERE video_id = ?
494         ORDER BY rally_number
495         """, (video_id,))
496         for row in cursor.fetchall():
497             rallies.append(TableTennisRally(
498                 video_id=row['video_id'],
499                 rally_number=row['rally_number'],
500                 start_time=row['start_time'],
501                 end_time=row['end_time'],
502                 score_before=row['score_before'],

```

```

503             score_after=row['score_after'],
504             shots_count=row['shots_count'],
505             ending_reason=row['ending_reason']
506         ))
507     return rallies
508
509     def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
510         """Return chronologically-ordered (start_time, end_time) tuples for a video,
511         regardless of sport. This is the sport-agnostic view used to export
512         indexer-ready evaluation/training labels.
513
514         Sports that have no annotation table yet (e.g. tabletennis, pickleball)
515         yield an empty list rather than raising.
516         """
517         mapping = SEGMENT_TABLES.get(sport)
518         if mapping is None:
519             return []
520         table, order_by = mapping # both come from the trusted map above, never user
input
521         with self._get_connection() as conn:
522             cursor = conn.cursor()
523             cursor.execute(
524                 f"SELECT start_time, end_time FROM {table} "
525                 f"WHERE video_id = ? ORDER BY {order_by}",
526                 (video_id,),
527             )
528             return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
529
530     def update_status(self, video_id: str, status: CurationStatus) -> bool:
531         with self._get_connection() as conn:
532             cursor = conn.cursor()
533             cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
534 (status.value, video_id))
535             conn.commit()
536             return cursor.rowcount > 0
537
538     def delete_video(self, video_id: str) -> bool:
539         """Deletes a video and cascading child entries."""
540         with self._get_connection() as conn:
541             cursor = conn.cursor()
542             # Execute cascade deletes manually just in case SQLite foreign key support
is not active
543             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
544 (video_id,))
545             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
546             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
547 (video_id,))
548             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
549 (video_id,))
550             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
551 (video_id,))
552             cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
553             conn.commit()
554             return cursor.rowcount > 0
555

```

13. user (2026-06-06T06:45:27.291Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple

```

```

8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.
41            rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43            if not rows:
44                print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                return
46
47            data = [[sport, status, cnt] for sport, status, cnt in rows]
48            print("\n=== Dataset Ingestion Status ===")
49            print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50            print()
51
52        def import_local(self, args):
53            """Manually registers a local video and annotation file into the database."""
54            sport_str = args.sport.lower()
55            if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57                sys.exit(1)
58
59            # Validate video path
60            if not os.path.exists(args.video_path):
61                print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63            # Validate annotation file
64            if not os.path.exists(args.annotations_path):
65                print(f"Error: Annotations file '{args.annotations_path}' not found.")
66                sys.exit(1)
67
68            match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]

```

```

69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f" - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata
109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )
122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)

```

```

131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)
137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f" - Video ID: {video_id}")
140         print(f" - Sport: {sport_str}")
141         print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142         print(f" - Registered {count} annotated segments.\n")
143
144     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
145         """Parses simple JSON or CSV annotations files."""
146         parsed_items = []
147         ext = os.path.splitext(filepath)[1].lower()
148
149         if ext == '.json':
150             with open(filepath, 'r', encoding='utf-8') as f:
151                 data = json.load(f)
152
153             if not isinstance(data, list):
154                 raise ValueError("JSON file must contain a list of objects.")
155
156             # Uses the same safe casts as the CSV path: a missing/blank required
157             # time raises a clear ValueError (caught by the caller) instead of a
158             # bare KeyError, and optional ints degrade to sensible defaults.
159             for idx, obj in enumerate(data):
160                 if sport == "badminton":
161                     parsed_items.append(BadmintonRally(
162                         video_id=video_id,
163                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
164                         start_time=_safe_float(obj.get("start_time")),
165                         end_time=_safe_float(obj.get("end_time")),
166                         score_before=obj.get("score_before"),
167                         score_after=obj.get("score_after"),
168                         shots_count=_safe_int(obj.get("shots_count"), None),
169                         ending_reason=obj.get("ending_reason")
170                     ))
171                 elif sport == "tennis":
172                     parsed_items.append(TennisPoint(
173                         video_id=video_id,
174                         point_number=_safe_int(obj.get("point_number"), idx + 1),
175                         start_time=_safe_float(obj.get("start_time")),
176                         end_time=_safe_float(obj.get("end_time")),
177                         set_score=obj.get("set_score"),
178                         game_score=obj.get("game_score"),
179                         server=obj.get("server"),
180                         ending_type=obj.get("ending_type")
181                     ))
182                 elif sport == "cricket":
183                     parsed_items.append(CricketDelivery(
184                         video_id=video_id,
185                         over=_safe_int(obj.get("over"), 0),
186                         ball=_safe_int(obj.get("ball"), 0),
187                         start_time=_safe_float(obj.get("start_time")),
188                         end_time=_safe_float(obj.get("end_time")),
189                         batsman=obj.get("batsman"),
190                         bowler=obj.get("bowler"),
191                         runs=_safe_int(obj.get("runs"), None),
192                         wicket=bool(obj.get("wicket", False)),
193                         extras=_safe_int(obj.get("extras"), 0)

```

```

194         ))
195     elif sport == "pickleball":
196         parsed_items.append(PickleballRally(
197             video_id=video_id,
198             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
199             start_time=_safe_float(obj.get("start_time")),
200             end_time=_safe_float(obj.get("end_time")),
201             score_before=obj.get("score_before"),
202             score_after=obj.get("score_after"),
203             shots_count=_safe_int(obj.get("shots_count"), None),
204             ending_reason=obj.get("ending_reason")
205         ))
206     elif sport == "tabletennis":
207         parsed_items.append(TableTennisRally(
208             video_id=video_id,
209             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
210             start_time=_safe_float(obj.get("start_time")),
211             end_time=_safe_float(obj.get("end_time")),
212             score_before=obj.get("score_before"),
213             score_after=obj.get("score_after"),
214             shots_count=_safe_int(obj.get("shots_count"), None),
215             ending_reason=obj.get("ending_reason")
216         ))
217     elif ext == '.csv':
218         with open(filepath, 'r', encoding='utf-8') as f:
219             reader = csv.DictReader(f)
220             # Normalize headers
221             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
222
223         for idx, row in enumerate(reader):
224             if sport == "badminton":
225                 parsed_items.append(BadmintonRally(
226                     video_id=video_id,
227                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
228                     start_time=_safe_float(row.get("start_time")),
229                     end_time=_safe_float(row.get("end_time")),
230                     score_before=row.get("score_before"),
231                     score_after=row.get("score_after"),
232                     shots_count=_safe_int(row.get("shots_count"), None),
233                     ending_reason=row.get("ending_reason")
234                 ))
235             elif sport == "tennis":
236                 parsed_items.append(TennisPoint(
237                     video_id=video_id,
238                     point_number=_safe_int(row.get("point_number"), idx + 1),
239                     start_time=_safe_float(row.get("start_time")),
240                     end_time=_safe_float(row.get("end_time")),
241                     set_score=row.get("set_score"),
242                     game_score=row.get("game_score"),
243                     server=row.get("server"),
244                     ending_type=row.get("ending_type")
245                 ))
246             elif sport == "cricket":
247                 parsed_items.append(CricketDelivery(
248                     video_id=video_id,
249                     over=_safe_int(row.get("over"), 0),
250                     ball=_safe_int(row.get("ball"), 0),
251                     start_time=_safe_float(row.get("start_time")),
252                     end_time=_safe_float(row.get("end_time")),
253                     batsman=row.get("batsman"),
254                     bowler=row.get("bowler"),
255                     runs=_safe_int(row.get("runs"), None),
256                     wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],

```

```

257         extras=_safe_int(row.get("extras"), 0)
258     ))
259     elif sport == "pickleball":
260         parsed_items.append(PickleballRally(
261             video_id=video_id,
262             rally_number=_safe_int(row.get("rally_number"), idx + 1),
263             start_time=_safe_float(row.get("start_time")),
264             end_time=_safe_float(row.get("end_time")),
265             score_before=row.get("score_before"),
266             score_after=row.get("score_after"),
267             shots_count=_safe_int(row.get("shots_count"), None),
268             ending_reason=row.get("ending_reason")
269         ))
270     elif sport == "tabletennis":
271         parsed_items.append(TableTennisRally(
272             video_id=video_id,
273             rally_number=_safe_int(row.get("rally_number"), idx + 1),
274             start_time=_safe_float(row.get("start_time")),
275             end_time=_safe_float(row.get("end_time")),
276             score_before=row.get("score_before"),
277             score_after=row.get("score_after"),
278             shots_count=_safe_int(row.get("shots_count"), None),
279             ending_reason=row.get("ending_reason")
280         ))
281     else:
282         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
283
284     return parsed_items
285
286 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
287     """Download one video at best resolution, retrying transient failures.
288
289     A below-min-height result almost always means YouTube rate-limited the
290     request and served a low fallback (not that the video lacks HD), so we
291     retry with a growing backoff. Returns (DownloadResult, None) on success,
292     or (None, failure_reason) once retries are exhausted. A below-min file is
293     discarded each attempt so it never lands in the DB.
294     """
295     attempts = max(1, args.retries + 1)
296     reason = "unknown error"
297     for i in range(attempts):
298         result = download_best_video(
299             url, dest_path,
300             max_height=max_height, max_size_mb=max_size_mb,
301             cookies_from_browser=args.cookies_from_browser,
302             cookies_file=args.cookies,
303             player_client=args.player_client,
304         )
305         if result.success and (result.height is None or result.height >=
args.min_height):
306             return result, None
307
308         if not result.success:
309             reason = result.error or "unknown error"
310         else:
311             reason = (f"got {result.resolution_label} (< min-height
{args.min_height})p) ? "
312                 f"YouTube served a low format (rate-limited?)")
313         try:
314             os.remove(result.path)
315         except OSError:
316             pass
317
318     if i < attempts - 1:

```

```

319         backoff = args.retry_backoff * (i + 1)
320         print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
321         time.sleep(backoff)
322     return None, reason
323
324 def fetch(self, args):
325     """(Re)download videos at best available resolution, updating the DB in place.
326
327     This is the in-place upgrade path: rows, rally annotations, and curation
328     status are all preserved ? only the video file plus its `local_path`,
329     `resolution`, and `fps` are refreshed. By default it skips videos that
330     already have a local file at or above `--min-height`, so re-runs are cheap
331     and resumable; `--force` re-fetches everything.
332     """
333     storage = self.config.get("video_storage", {})
334     local_dir = storage.get("local_dir", "./data/videos")
335     max_download_size_mb = storage.get("max_download_size_mb")
336     max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
337     os.makedirs(local_dir, exist_ok=True)
338
339     commercial_only = not args.include_noncommercial
340     if args.status == "all":
341         statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
342     else:
343         statuses = [CurationStatus(args.status)]
344
345     supported = self.config.get("supported_sports", ["badminton"])
346     sport_strs = [args.sport] if args.sport else supported
347
348     # Collect target videos.
349     videos = []
350     for sport_str in sport_strs:
351         try:
352             sport = SportType(sport_str.lower())
353         except ValueError:
354             print(f"Warning: skipping unknown sport '{sport_str}'.")
355             continue
356         for status in statuses:
357             videos.extend(self.db.get_videos_by_status(status, sport))
358     if args.video_id:
359         videos = [v for v in videos if v.video_id == args.video_id]
360
361     if not videos:
362         print("\nNo matching videos to fetch.\n")
363         return
364
365     results = [] # (video_id, action, detail)
366     for video in videos:
367         if commercial_only and not video.is_commercial:
368             results.append((video.video_id, "skip", "non-commercial"))
369             continue
370         if not video.video_url or "watch?v=example" in video.video_url:
371             # Either no URL, or the parser's placeholder (the ShuttleSet catalog
372             # had no URL for this match) ? nothing to download.
373             results.append((video.video_id, "skip", "no real source URL"))
374             continue
375
376         dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
377
378         # Skip if we already have a good-enough local file (unless --force).
379         if not args.force and os.path.exists(dest_path):
380             _, height, _ = probe_resolution_fps(dest_path)
381             if height is not None and height >= args.min_height:

```

```

382         results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
383         continue
384
385     print(f"Fetching {video.video_id} ({video.match_name})...")
386     result, fail_reason = self._fetch_one_with_retry(
387         video.video_url, dest_path, args, max_height, max_download_size_mb)
388     if fail_reason is not None:
389         results.append((video.video_id, "FAIL", fail_reason))
390         if args.sleep:
391             time.sleep(args.sleep)
392         continue
393
394     # Update only the file-derived fields; preserve everything else (UPSERT
395     # mutates the row in place, leaving rallies intact).
396     video.local_path = result.path
397     if result.resolution_label:
398         video.resolution = result.resolution_label
399     if result.fps:
400         video.fps = result.fps
401     self.db.insert_video(video)
402
403     detail = (f"{result.resolution_label or '?'}"
404              f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
405     results.append((video.video_id, "fetched", detail))
406     if args.sleep:
407         time.sleep(args.sleep)
408
409     print("\n=== Fetch summary ===")
410     print(tabulate([[vid, action, detail] for vid, action, detail in results],
411                   headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
412     fetched = sum(1 for _, a, _ in results if a == "fetched")
413     failed = [r for r in results if r[1] == "FAIL"]
414     print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
results if a=='have'))}    "
415          f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}}    Failed :
{len(failed)}")
416     if failed:
417         print("Failures (left unchanged in DB):")
418         for vid, _, detail in failed:
419             print(f"    - {vid}: {detail}")
420     print()
421
422     def export_eval_set(self, args):
423         """Export curated annotations as indexer-ready eval/training sets.
424
425         Emits, per qualifying video, a `//.csv` in the
426         indexer's `start,end` ground-truth format (decimal seconds), plus a single
427         `/manifest.json` that pairs each CSV with its video file. The
428         manifest is the contract the downstream eval/training driver reads: the
429         indexer keys videos by `*file MD5*`, not by our `video_id`, so it must run
430         against the exact `local_path`/`video_url` recorded here.
431
432         By default only CURATED + commercially-safe videos are exported, mirroring
433         the license gate already enforced elsewhere in the toolchain.
434         """
435         out_dir = args.out_dir
436         commercial_only = not args.include_noncommercial
437         statuses = [CurationStatus.CURATED]
438         if args.include_staging:
439             statuses.append(CurationStatus.STAGING)
440
441         supported = self.config.get("supported_sports", ["badminton"])
442         sport_strs = [args.sport] if args.sport else supported

```

```

443
444 manifest = []
445 skipped_noncomm = 0
446 skipped_empty = 0
447
448 for sport_str in sport_strs:
449     try:
450         sport = SportType(sport_str.lower())
451     except ValueError:
452         print(f"Warning: skipping unknown sport '{sport_str}'.")
453         continue
454
455     videos = []
456     for status in statuses:
457         videos.extend(self.db.get_videos_by_status(status, sport))
458
459     for video in videos:
460         if commercial_only and not video.is_commercial:
461             skipped_noncomm += 1
462             continue
463
464         intervals = self.db.get_segment_intervals(video.video_id, sport)
465         if not intervals:
466             skipped_empty += 1
467             continue
468
469         sport_dir = os.path.join(out_dir, sport.value)
470         os.makedirs(sport_dir, exist_ok=True)
471         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
472         with open(csv_path, "w", newline="", encoding="utf-8") as f:
473             writer = csv.writer(f)
474             writer.writerow(["start", "end"])
475             for start, end in intervals:
476                 writer.writerow([start, end])
477
478         manifest.append({
479             "video_id": video.video_id,
480             "sport": sport.value,
481             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
482             "local_path": video.local_path,
483             "video_url": video.video_url,
484             "match_name": video.match_name,
485             "fps": video.fps,
486             "resolution": video.resolution,
487             "license_type": video.license_type.value,
488             "is_commercial": video.is_commercial,
489             "status": video.status.value,
490             "num_segments": len(intervals),
491         })
492
493     os.makedirs(out_dir, exist_ok=True)
494     manifest_doc = {
495         "commercial_only": commercial_only,
496         "statuses": [s.value for s in statuses],
497         "total_videos": len(manifest),
498         "total_segments": sum(m["num_segments"] for m in manifest),
499         "eval_sets": manifest,
500     }
501     manifest_path = os.path.join(out_dir, "manifest.json")
502     with open(manifest_path, "w", encoding="utf-8") as f:
503         json.dump(manifest_doc, f, indent=2)
504
505     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
506     if manifest:
507         summary = [[m["sport"], m["video_id"], m["num_segments"],

```

```

508                 "YES" if m["is_commercial"] else "NO",
509                 os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
510             for m in manifest]
511         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
512         print(f"\nVideos exported : {len(manifest)}")
513         print(f"Total segments : {manifest_doc['total_segments']}")
514         if skipped_noncomm:
515             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
516         if skipped_empty:
517             print(f"Skipped (no annotations) : {skipped_empty}")
518         print(f"Manifest : {manifest_path}\n")
519
520     def curate(self):
521         """Interactive loop to review and curate staging video annotations."""
522         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
523         if not staging_videos:
524             print("\nNo staging video annotations to review.\n")
525             return
526
527         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
528         table_data = []
529         for idx, video in enumerate(staging_videos):
530             table_data.append([
531                 idx + 1,
532                 video.video_id,
533                 video.match_name,
534                 video.sport.value,
535                 video.license_type.value,
536                 "YES" if video.is_commercial else "NO"
537             ])
538
539         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
540         print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
541
542         choice = input("> ").strip()
543         if choice.lower() == 'q':
544             return
545
546         try:
547             idx = int(choice) - 1
548             if idx < 0 or idx >= len(staging_videos):
549                 print("Invalid choice.")
550                 return
551             self._curate_video_interactive(staging_videos[idx])
552         except ValueError:
553             print("Invalid input.")
554
555     def _curate_video_interactive(self, video: VideoMetadata):
556         """Displays details of a specific staging video and prompts for curation
action."""
557
558         # Load and count events
559         rallies = []
560         pts = []
561         delivs = []
562         if video.sport == SportType.BADMINTON:
563             rallies = self.db.get_badminton_rallies(video.video_id)
564         elif video.sport == SportType.PICKLEBALL:
565             rallies = self.db.get_pickleball_rallies(video.video_id)
566         elif video.sport == SportType.TABLE_TENNIS:

```

```

567         rallies = self.db.get_tabletennis_rallies(video.video_id)
568     elif video.sport == SportType.TENNIS:
569         pts = self.db.get_tennis_points(video.video_id)
570     elif video.sport == SportType.CRICKET:
571         delivs = self.db.get_cricket_deliveries(video.video_id)
572
573     while True:
574         print("\n===== REVIEWING VIDEO =====")
575         print(f"Match Name      : {video.match_name}")
576         print(f"Video ID       : {video.video_id}")
577         print(f"Sport          : {video.sport.value}")
578         print(f"Video URL      : {video.video_url or 'N/A'}")
579         print(f"Local Path     : {video.local_path or 'N/A'}")
580         print(f"License       : {video.license_type.value} ({video.license_url or 'no
URL'})")
581         print(f"Commercial OK: {video.is_commercial}")
582
583         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
584             print(f"Total Rallies: {len(rallies)}")
585             if rallies:
586                 print("Sample rallies:")
587                 sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
588                 print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
589             elif video.sport == SportType.TENNIS:
590                 print(f"Total Points : {len(pts)}")
591                 if pts:
592                     print("Sample points:")
593                     sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
594                     print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
595             elif video.sport == SportType.CRICKET:
596                 print(f"Total Balls  : {len(delivs)}")
597                 if delivs:
598                     print("Sample deliveries:")
599                     sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
600                     print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
601
602         print("\nChoose Curation Action:")
603         print(" [a] Approve (promote to CURATED)")
604         print(" [r] Reject (delete from database)")
605         print(" [p] Play a rally/point segment in browser")
606         print(" [k] Keep in Staging (do nothing and go back)")
607
608         action = input("> ").strip().lower()
609         if action == 'a':
610             self.db.update_status(video.video_id, CurationStatus.CURATED)
611             print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
612             break
613         elif action == 'r':
614             self.db.delete_video(video.video_id)
615             print(f"Successfully rejected and deleted '{video.match_name}'.\n")
616             break
617         elif action == 'k':
618             print("Curation state unchanged.\n")
619             break
620         elif action == 'p':
621             self._play_segment_interactive(video, rallies, pts, delivs)
622         else:
623             print("Invalid action selected.")

```

```

624
625     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
626         import webbrowser
627
628         # Get start/end based on user input
629         start_time = None
630         end_time = None
631         label = "segment"
632
633         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
634             if not rallies:
635                 print("No rallies to play.")
636                 return
637             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
638             try:
639                 rally_num = int(idx_str)
640                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
641                 if rally:
642                     start_time = rally.start_time
643                     end_time = rally.end_time
644                     label = f"Rally {rally_num}"
645             except ValueError:
646                 pass
647         elif video.sport == SportType.TENNIS:
648             if not pts:
649                 print("No points to play.")
650                 return
651             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
652             try:
653                 pt_num = int(idx_str)
654                 pt = next((p for p in pts if p.point_number == pt_num), None)
655                 if pt:
656                     start_time = pt.start_time
657                     end_time = pt.end_time
658                     label = f"Point {pt_num}"
659             except ValueError:
660                 pass
661         elif video.sport == SportType.CRICKET:
662             if not delivs:
663                 print("No deliveries to play.")
664                 return
665             over_str = input("Enter Over number: ").strip()
666             ball_str = input("Enter Ball number: ").strip()
667             try:
668                 o_num = int(over_str)
669                 b_num = int(ball_str)
670                 d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
671                 if d:
672                     start_time = d.start_time
673                     end_time = d.end_time
674                     label = f"Over {o_num} Ball {b_num}"
675             except ValueError:
676                 pass
677
678         if start_time is None or end_time is None:
679             print("Segment not found or invalid input.")
680             return
681
682         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
683
684         # Check video location
685         play_url = None

```

```

686         if video.local_path and os.path.exists(video.local_path):
687             abs_path = os.path.abspath(video.local_path)
688             # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
689             play_url = f"file:/// {abs_path.replace(os.sep,
'/'')}#t={start_time},{end_time}"
690         elif video.video_url:
691             # YouTube URL: seek to start time
692             if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
693                 t_sec = int(start_time)
694                 sep = "&" if "?" in video.video_url else "?"
695                 play_url = f"{video.video_url}{sep}t={t_sec}s"
696             else:
697                 play_url = video.video_url
698
699         if play_url:
700             print(f"Opening browser: {play_url}")
701             webbrowser.open(play_url)
702         else:
703             print("Error: No local video file or remote URL available to play.")
704
705
706     def main():
707         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
708         subparsers = parser.add_subparsers(dest="command")
709
710         # Status command
711         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
712
713         # Curate command
714         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
715
716         # Import-local command
717         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
718         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
719         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
720         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
721         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
722         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
723         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
724         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
725         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
726
727         # Fetch command ? (re)download videos at best resolution, update DB in place
728         fetch_parser = subparsers.add_parser(
729             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
730         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
731         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
732         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
733                                     help="Which curation statuses to fetch (default: all)")
734         fetch_parser.add_argument("--min-height", type=int, default=720,
735                                     help="Skip videos already at/above this height unless

```

```

--force (default: 720)")
736         fetch_parser.add_argument("--max-height", type=int, default=None,
737                                     help="Cap download resolution (e.g. 1080); default: best
available")
738         fetch_parser.add_argument("--force", action="store_true",
739                                     help="Re-download even if a good-enough local file already
exists")
740         fetch_parser.add_argument("--include-noncommercial", action="store_true",
741                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
742         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
743                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
744                                     "A Premium login gives reliable best-resolution
downloads. "
745                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
746         fetch_parser.add_argument("--cookies", metavar="FILE",
747                                     help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
748         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
749                                     help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
750                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
751         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
752                                     help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
753         fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
754                                     help="Retries per video when a download fails or returns a
low format (default: 2)")
755         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
756                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
757
758         # Export command
759         export_parser = subparsers.add_parser(
760             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
761         export_parser.add_argument("--out-dir", default="data/eval_export",
762                                     help="Output directory for per-video CSVs and
manifest.json")
763         export_parser.add_argument("--sport",
764                                     help="Restrict to a single sport (default: all
supported_sports)")
765         export_parser.add_argument("--include-staging", action="store_true",
766                                     help="Also export videos still in staging (default:
curated only)")
767         export_parser.add_argument("--include-noncommercial", action="store_true",
768                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
769
770         args = parser.parse_args()
771         cli = SportsDataCLI()
772
773         if args.command == "status":
774             cli.status()
775         elif args.command == "curate":
776             cli.curate()
777         elif args.command == "import-local":
778             cli.import_local(args)
779         elif args.command == "export":

```

```

780         cli.export_eval_set(args)
781     elif args.command == "fetch":
782         cli.fetch(args)
783     else:
784         # Default: print help
785         parser.print_help()
786
787 if __name__ == "__main__":
788     main()
789

```

14. assistant (2026-06-06T06:45:33.713Z)

I now have the full picture of the eval/provider/exporter contract. Let me read the two indexer docs to capture the intended provider contract, plus the badminton schema and the harness's `get\_predictions` to nail units. Then I'll inspect the actual Roora exports to ground the adapter.

15. user (2026-06-06T06:45:34.656Z)

```

1      # Golden-Set Subproject ? Design & Implementation Overview
2
3      **Status:** In progress · **Date:** 2026-05-30
4      **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6      This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8      > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10     ---
11
12     ## Architecture in one picture
13
14     ```
15         video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]  ground truth
16                 ?           ?           ?                               ?
17         FileProvider  HumanVendor  Automated(oracle)                   ?
18         (GS-1)         (GS-3)       (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19
20     production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]  predictions
21
22                                     regress(video, tier) (GS-2)
23     compares the two
24     ```
25     **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27     ---

```

```

28
29     ## PR breakdown
30
31     | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32     |---|---|---|---|---|
33     | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34     | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35     | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36     | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37     | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39     Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41     ---
42
43     ## Slice details
44
45     ### GS-1 ? Provider registry + FileProvider ? (this PR)
46     - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate()` convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47     - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48     - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49     - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51     ### GS-2 ? Regression runner (next)
52     `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54     ### GS-3 ? HumanVendorProvider (after your vendor pick)
55     A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
56
57     ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
58     An independent strong labeler for the fast/CI tier. **Must be a different model lineage
than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
own.
59
60     ### GS-5 ? Regression triggers
61     Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the
"surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so
legitimate production improvements can rebaseline instead of raising false regressions.
62
63     ---
64
65     ## Conventions
66     - **Branch per slice**, one PR each, tests green before PR, full suite must stay green.

```

67 - New runtime deps: **none** through GS-2 (GS-3 may add a vendor SDK; GS-4 may reuse the existing `google-genai` for a Gemini oracle).
68 - All ground-truth video stays **licensed/owned** ? see the privacy section of the design doc.
69

16. user (2026-06-06T06:45:36.040Z)

```
1      # Golden-Set Generation & Vendor Process (v1)
2
3      Status: Design v1 . Date: 2026-05-30 . Vendor shortlist added 2026-05-30
($6)
4      Owner decisions captured: licensed/owned video only . rally `[start,end)` depth .
tiered turnaround (fast automated + slow human) . vendor bake-off . Bangalore/India-local
vendor preference
5
6      This document defines how we produce golden-set ground truth (hand-verified rally
boundaries) for the badminton highlights product, how we select annotation vendors, and how
that same machinery powers on-demand / surprise regression testing of the production tool.
7
8      > Companion docs: the scoring mechanics live in [EVALUATION.md](EVALUATION.md); the
annotated-corpus rationale is [ADR-002](DECISIONS.md); the decision record for this process is
ADR-006. Video-capture quality requirements (a dependency of trustworthy golden sets) are
tracked separately in [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md).
9
10     > Update (2026-06-02): $6 shortlist done; the founder selected Roora (Bengaluru)
for the initial paid pilot ? a single-vendor pilot first, not the full bake-off below.
Roora's site/capability/location are verified; quality is TBD via the pilot. Step 0 (the
annotation guideline) is authored ? collector repo `docs/annotation/` (`ROORA_BRIEF.md` rally
CSV schema + `ANNOTATION_GUIDE.md` illustrated guide + intake-manifest & NDA templates). Pilot
scope is 3 sports (badminton, pickleball, table tennis); the collector's import-local
ingestion for all three is built. Still pending: the internal gold key and the vendor-
dependent GS-3 `HumanVendorProvider` (GS-1/GS-2/GS-4 already landed ? see
[GOLDEN_SET_IMPLEMENTATION.md](GOLDEN_SET_IMPLEMENTATION.md)).
11
12     ---
13
14     ## 1. The keystone insight: one scorer, three jobs
15
16     A vendor's golden-set output is a list of rally `[start,end)` intervals ? structurally
identical to a prediction from our pipeline. So the exact same deterministic scorer in
`backend/eval/metrics.py` (`interval_iou` ? `match_intervals` ? `score`) grades all three of:
17
18     1. Product vs. ground truth ? the precision/recall/F1/IoU we already report
(`EVALUATION.md`).
19     2. Vendor vs. our internal answer key ? the quality gate during vendor selection.
20     3. Annotator vs. annotator ? inter-annotator agreement (IAA), our proxy for "is this
even a well-defined task?"
21
22     This symmetry is what makes the whole system small, auditable, and extensible. No new
scoring code is needed ? only a new source of intervals (the vendor).
23
24     ---
25
26     ## 2. Why this also solves the privacy nuance
27
28     The monetization goal (hosted API) carries a "someone views the user's video" concern.
This process sidesteps it by construction:
29
30     - Golden sets are built only from footage we license or own ? starting with the
founder's constantly-growing personal collection of match videos, plus ShuttleSet (ADR-003)
and any purpose-recorded clips. No end-user uploads ever flow to a vendor.
31     - Regression tests therefore run against a representative held-out corpus, not
literal user content.
32     - All video egress to any third party passes through a single provider boundary
```

(§4), so the security controls (DPA, no-reuse, delete-on-delivery, access logging) live in exactly one place.

33

34 > **Honest tradeoff:** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore **strong evidence**, not a **guarantee**, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see VIDEO_RECORDING_GUIDELINES.md, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

35

36 ---

37

38 ## 3. Selecting vendors ? the bake-off

39

40 ### Step 0 ? Write the annotation guideline (highest-leverage artifact)

41 Before contacting anyone, write a precise, example-rich definition of **what a rally is**:

42 - **Start trigger:** e.g. first serve motion vs. shuttle leaving the racket ? pick one, show clips.

43 - **End trigger:** shuttle hits floor/net/out, or umpire call.

44 - **Edge cases:** lets/replays, service faults, between-point walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.

45

46 Rally boundaries are genuinely subjective; **ambiguity** here is the #1 cause of bad golden sets. This guideline belongs conceptually with the per-sport `SportProfile``, so it generalizes to tennis/cricket later.

47

48 > **Done (2026-06-02):** authored as the Roora **brief + illustrated guide** in the collector repo `docs/annotation/``. Defines the rally CSV schema and the three golden rules ? ignore all dead time; include **every** rally (incl. service faults & lets); a shot = one racket/paddle/bat contact ? plus observational `ending_reason`` and per-sport detail incl. **singles & doubles** for badminton & pickleball.

49

50 ### Step 1 ? Build a small internal gold key

51 One trusted expert (initially: the founder) hand-annotates **5?10** licensed clips to near-perfect. This is the answer key vendors are graded against ? kept private during the bake-off.

52

53 ### Step 2 ? Run the bake-off (parallel pilot)

54 Send the **same** pilot clips to **2?3** shortlisted vendors (see §6, favoring Bangalore/India), withholding the key. Grade every submission with **our own harness**:

55

Dimension	How we measure it	Tool
Accuracy	vendor CSV vs. gold key ? P/R/F1, mIoU, boundary error	<code>eval.score()</code>
Consistency	2 annotators per vendor ? pairwise IoU (IAA)	<code>interval_iou`</code>
Cost	quoted ? or \$ per video-minute	vendor quote
Turnaround	actual vs. promised, per tier	observed
Ops friction	file-drop vs. API; compliance with our CSV format	<code>annotations.load_annotations()</code>
Security	DPA signed; DPDP/ISO posture; delete-on-delivery	contract review

64

65 ### Step 3 ? Decide on measured quality-per-dollar

66 The bake-off numbers **are** the decision ? not sales decks. Low IAA at a vendor means either a weak vendor **or** a vague guideline (Step 0); investigate before blaming the vendor.

67

68 ### Step 4 ? Onboard the winner(s)

69 Sign DPA + the security controls in §4; codify the guideline as their spec; agree SLAs for both tiers (§5).

70

71 ---

72

73 ## 4. The extensible architecture ? an `AnnotationProvider`` registry

74

75 This codebase already uses pluggable registries (validators, segmenters, AI providers,

sports). The idiomatic move is the same shape for annotation sourcing:

```
76
77     ```python
78     class AnnotationProvider(ABC):
79         def submit(self, video_ref, guideline, sport) -> str:           # returns job_id
80             ...
81         def poll(self, job_id) -> str:                                   #
82             ...
83         def fetch(self, job_id) -> List[Interval]:                     # canonical
84             ...
85     ```
86
87     `fetch()` returns the same `List[Interval]` that `annotations.load_annotations()`
88     already produces, so everything downstream is unchanged.
89
90     Three implementations cover today plus both tiers:
91
92     | Provider | Role | Tier |
93     |---|---|---|
94     | `FileProvider` | Reads a pre-made CSV (formalizes today's manual / ShuttleSet flow) |
95     ? |
96     | `HumanVendorProvider` | The bake-off winner; authoritative ground truth, corpus
97     growth, deep audits | Slow (days) |
98     | `AutomatedProvider` | An independent strong model labels rallies programmatically |
99     Fast (hours) |
100
101     The tiers are just two providers behind one interface, selectable per call.
102
103     ---
104
105     ## 5. The regression automation (the explicit ask)
106
107     > "given a video, get a golden set created and verify its precision and recall against
108     the production tool"
109
110     This becomes a thin wrapper over machinery that already exists:
111
112     ```python
113     def regress(video, tier="fast", tolerance=0.05):
114         gt = annotation_registry.get(tier).fetch(                        # NEW: the provider
115             annotation_registry.get(tier).submit(video, GUIDELINE, sport))
116         predictions = process(video)                                     # existing production
117     pipeline ? DB
118         s = eval.score(predictions, gt)                                  # existing metrics.py,
119         unchanged
120         regressed = (s.recall < baseline[video].recall - tolerance or
121             s.precision < baseline[video].precision - tolerance)
122         if regressed: alert(video, s, baseline[video])
123         return s
124     ```
125
126     Only one new component (the provider). `run_eval.py`, `harness.py`, and `metrics.py`
127     are reused verbatim. Snapshot a `baseline.json` per corpus video so "regression" is a measured
128     delta, not a vibe.
129
130     Triggers (all feed the same `regress()`): CI on pipeline changes · on-demand (the
131     "surprise" case) · scheduled cron · pre-release gate.
132
133     ### ?? The oracle problem (a real correctness trap)
134
135     The fast-tier `AutomatedProvider` must be a different model lineage than production.
136     If production becomes the offline ActionFormer segmenter (ADR-004) and the oracle is also
137     ActionFormer, they share blind spots ? the test shows green while both are wrong. Pairings that
138     stay honest:
```

```

125
126     - production = offline learned segmenter ? fast-tier oracle = **paid Gemini full-video**
(different lineage).
127     - production = Gemini hybrid ? fast-tier oracle = a **different VLM** or the slow human
tier.
128
129     Treat the fast tier as a **smoke alarm**, not ground truth: any fast-tier "regression"
**escalates to the slow human tier** for confirmation before it blocks a release.
130
131     ---
132
133     ## 6. Vendor shortlist (Bangalore/India-favoring)
134
135     > **Researched 2026-05-30 via a lean manual pass** (targeted web search + direct
site/source verification ? NOT the heavy automated workflow, which failed). **Confidence is
labelled per row.** **No pricing is listed because none could be verified** ? every vendor in
this space quotes per-project, so ?/clip is an offline-conversation item (see §6.3). Treat
single-source rows as *leads to verify on the call*, not vetted recommendations.
136
137     ### 6.1 Tier-1 candidates ? verified India presence + video capability (talk to these
first)
138
139     | Vendor | Location (verified) | Video / temporal labeling | Notes for the call |
Confidence |
140     |---|---|---|---|---|
141     | **iMerit** | **Bengaluru** office (A101 & A118, 43 Residency Rd, 560025; tech hub
opened Oct 2022); also Kolkata HQ + US | Yes ? image/video/text/audio; explicit Sports &
Biomechanics domain on their site | Largest & most enterprise-ready (5,500+ annotators); has a
real **Bangalore** footprint for in-person coordination; ask for a sports/temporal-segmentation
reference + a paid pilot. Higher-touch ? likely higher cost. | **High** |
142     | **TELUS International AI** (ex-**Playment**) | Playment was **Bangalore-founded**
(2015); acquired by TELUS 2021, team retained | Yes ? built on Playment's 2D/3D
image+video+LiDAR CV pipeline | Strong CV/video pedigree, Bangalore roots; but now a large
global BPO ? confirm they'll take a **small startup pilot** and who the India delivery contact
is. | **High** (origin) / **Med** (current India ops shape) |
143     | **Taskmonk** | **Bengaluru** | Yes ? "Video" listed as a labeling modality; platform
*** managed-service workforce | Mid-size, India-local, does both tooling and service ?
potentially the best **cost/agility** fit for a startup pilot. Verify temporal/event labeling
specifically (their heritage is e-commerce catalog tagging). | **Med-High** |
144
145     ### 6.2 Tier-2 ? India video-annotation services, location/specialts to confirm on
contact
146
147     | Vendor | Stated location | Note | Confidence |
148     |---|---|---|---|
149     | **Infols** | India (Kerala-based per third parties) | Targets startups/SMBs;
image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate | **Med** |
150     | **Analytics** | India + USA (since 2018) | General data-annotation incl. video; widely
listed | **Med** |
151     | **Objectways** | HQ Scottsdale AZ; **India delivery in Coimbatore** (not Bangalore) |
Does video/LiDAR; India delivery but not Bangalore-local | **Med** |
152     | **Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, `rooragr.com`) |
Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot
(2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**. |
**Verified** (capability) · pilot pending |
153     | **Tika Data** / **Zuru** | Claimed **Bangalore** (single-source listicle each) |
Smaller/startup annotation shops ? unverified beyond one directory mention; verify they exist &
do temporal video before investing time | **Low** |
154
155     ### 6.3 Explicitly NOT verified / out of scope
156     - **Pricing for everyone** ? all quote-only; do not assume. Get per-video-minute (or
per-rally) quotes during the offline conversations.
157     - **Karya (Bengaluru)** ? confirmed local and reputable, but **speech/text/translation
only, no video** ? **excluded** for our use case.
158     - **Sports/temporal track record** ? beyond iMerit's stated "Sports & Biomechanics"

```

domain, vendors' specific *rally-boundary / temporal-event* experience is unconfirmed marketing copy; probe with the pilot (§3).

159 - **DPDP / DPA / ISO posture** ? not verified per-vendor; make it a checklist item in the bake-off (§3 Step 2 security row).

160

161 ### 6.4 Recommendation

162 For the **offline conversations**, start with **iMerit** (Bangalore-local, enterprise QA, sports domain) and **Taskmonk** (Bangalore-local, likely more startup-friendly cost/agility) as the two primary contacts, with **TELUS/Playment** as a third if you want a heavyweight CV pedigree. Run the §3 bake-off across whichever 2?3 agree to a paid pilot, and decide on measured quality-per-rupee. **Verify every "Confidence: Med/Low" claim on the call before committing.**

163

164 > **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local, startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot doesn't clear the §3/§4 quality bar.

165

166 > Sources (manual pass, 2026-05-30): iMerit Bengaluru hub ? imerit.net press / PRNewswire (Oct 2022) + contact listings; Playment/TELUS ? BusinessWire & Inc42 (2021 acquisition, Bangalore-founded); Taskmonk & Karya locations ? company sites + AnalyticsIndiaMag / IndiaAI; others ? AnalyticsIndiaMag / Ziloservices / vendor sites. Pricing intentionally omitted (unverifiable).

167

168 ---

169

170 ## 7. Suggested phasing

171

	Phase	Deliverable	Depends on	
	---	---	---	
174	0	Annotation guideline + internal gold key (5?10 clips)	nothing ? can start now	
175	1	Vendor bake-off ? pick winner(s)	§6 research + guideline	
176	2	`AnnotationProvider` registry + `FileProvider` + `HumanVendorProvider`	winner onboarded	
177	3	`AutomatedProvider` (independent oracle) + `regress()` + baseline snapshots		
178	4	Wire surprise triggers (CI / on-demand / scheduled)		

179

180 Phase 0 is unblocked today and is the highest-leverage step regardless of vendor choice.

181

182 > **Status (2026-06-02):** Phase 0 guideline ? authored (collector `docs/annotation/`); internal gold key still pending. Phase 1 is running as a **single-vendor Roora pilot** across **3 sports** (badminton, pickleball, table tennis) rather than a multi-vendor bake-off. Phase 2's `FileProvider` (GS-1) and Phase 3's `regress()` + automated oracle (GS-2/GS-4) have already landed; the main remaining slice is the **vendor-dependent `HumanVendorProvider`** (GS-3), with trigger-wiring (GS-5) after it.

183

184 ---

185

186 ## 8. Open questions

187 1. **Guideline authorship** ? who owns the canonical rally definition, and is one expert's key enough, or do we want 2-expert consensus for the answer key itself?

188 2. **Corpus representativeness** ? how well does the owned/licensed collection mirror eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)

189 3. **Baseline drift** ? when production legitimately improves, how/when do we re-snapshot `baseline.json` so old baselines don't raise false regressions?

190 4. **Fast-tier oracle cost** ? per-video Gemini cost at regression frequency (see MONETIZATION_AUDIT.md §4 for the per-video estimate).

191

17. user (2026-06-06T06:45:36.358Z)

```

1 from pydantic import BaseModel, Field, model_validator
2 from typing import Optional
3
4 class BadmintonRally(BaseModel):

```

```

5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds")
8         end_time: float = Field(..., description="End of rally in seconds")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11        shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12        ending_reason: Optional[str] = Field(None, description="Reason the rally ended (e.g.
'kill', 'fault', 'out')")
13
14        @model_validator(mode='after')
15        def validate_timestamps(self) -> 'BadmintonRally':
16            if self.start_time < 0:
17                raise ValueError("start_time must be non-negative")
18            if self.end_time <= self.start_time:
19                raise ValueError("end_time must be strictly greater than start_time")
20            return self
21

```

18. assistant (2026-06-06T06:45:41.148Z)

Now let me verify the key facts empirically: the harness `get\_predictions` (for units), the CVAT XML structure, the COCO/VOC actual content, and the start_time/end_time vs frame-range hypothesis. Let me look at the actual Roora exports and the harness.

19. user (2026-06-06T06:45:42.124Z)

```

1        """Score the pipeline's detected rallies against ground-truth annotations.
2
3        Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4        so scoring is fast, deterministic, and free. Two prediction "stages" can be
5        scored:
6
7        - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8          the shuttle/person detector found, before AI confirmation.
9        - ``final``      ? confirmed play segments (``play_segments`` table). After the
10          AI handoff trimmed/validated boundaries.
11
12        Comparing the two tells you whether the weak link is *detection* (candidates
13        already miss rallies) or *segmentation* (candidates catch them but final is
14        wrong) ? see docs/EVALUATION.md.
15        """
16
17        import logging
18        from dataclasses import dataclass, field
19        from typing import List, Dict, Optional
20
21        from backend.database import Database
22        from backend.eval import metrics
23        from backend.eval.metrics import Interval, Score, MatchResult
24
25        logger = logging.getLogger(__name__)
26
27        STAGES = ("candidates", "final")
28
29
30        @dataclass
31        class StageReport:
32            stage: str
33            pred_intervals: List[Interval]
34            score: Score
35            match_result: MatchResult

```

```

36         pr_curve: List[Score] = field(default_factory=list)
37
38
39 @dataclass
40 class EvalReport:
41     video_id: str
42     iou_threshold: float
43     gt_intervals: List[Interval]
44     stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47 def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48     """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49     ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50     ivs.sort(key=lambda iv: iv[0])
51     return ivs
52
53
54 def get_predictions(db: Database, video_id: str, stage: str,
55                    detector: Optional[str] = None) -> List[Interval]:
56     """Fetch predicted rally intervals for one stage from the DB."""
57     if stage == "candidates":
58         rows = db.query_candidate_segments(video_id, detector=detector)
59     elif stage == "final":
60         rows = db.query_play_segments(video_id, {})
61     else:
62         raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63     return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67             stages: List[str], iou_threshold: float = 0.5,
68             detector: Optional[str] = None,
69             with_pr_curve: bool = False) -> EvalReport:
70     """Score the requested stages for one video against ground truth."""
71     report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                        gt_intervals=gt_intervals)
73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86     seconds = max(0, int(seconds))
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93     """JSON-serialisable view of an EvalReport."""
94     out = {
95         "video_id": report.video_id,
96         "iou_threshold": report.iou_threshold,
97         "num_ground_truth": len(report.gt_intervals),
98         "stages": {},
99     }
100     for stage, sr in report.stages.items():

```

```

101         out["stages"][stage] = {
102             "score": sr.score.as_dict(),
103             "pr_curve": [s.as_dict() for s in sr.pr_curve],
104         }
105     return out
106
107
108     def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109         """Human-readable console report for one video."""
110         lines = [
111             "=" * 64,
112             f" Rally evaluation ? video '{report.video_id}'",
113             f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114             "=" * 64,
115             f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116             f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117             "-" * 64,
118         ]
119         for stage in report.stages:
120             s = report.stages[stage].score
121             lines.append(
122                 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123                 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124                 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125             )
126             lines.append("=" * 64)
127
128         if show_failures:
129             for stage, sr in report.stages.items():
130                 misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132                 lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133                 for iv in misses:
134                     lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135                 lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136                 for iv in fps:
137                     lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139         return "\n".join(lines)
140

```

20. user (2026-06-06T06:45:43.302Z)

```

total 8
drwxr-xr-x 1 avidu 197609 0 Jun  6 11:24 .
drwxr-xr-x 1 avidu 197609 0 Jun  6 11:20 ..
drwxr-xr-x 1 avidu 197609 0 Jun  6 11:23 badminton
drwxr-xr-x 1 avidu 197609 0 Jun  6 11:24 badminton (1)
drwxr-xr-x 1 avidu 197609 0 Jun  6 11:24 badminton (2)
---badminton(2)---
total 232
drwxr-xr-x 1 avidu 197609      0 Jun  6 11:24 .
drwxr-xr-x 1 avidu 197609      0 Jun  6 11:24 ..
-rw-r--r-- 1 avidu 197609 232542 Jun  6 11:24 annotations.xml

```

21. assistant (2026-06-06T06:45:50.216Z)

The harness confirms predictions are read from DB rows with `start\_time`/`end\_time` as floats (harness.py:47-52), seconds. Now let me examine the CVAT XML structure and verify the start_time/end_time-vs-frame-range hypothesis empirically. Let me write a small Python probe to

parse the CVAT XML.

22. user (2026-06-06T06:45:55.269Z)

```
=== META/JOB ===
id = 4075945
size = 710
mode = interpolation
start_frame = 0
stop_frame = 21270
frame_filter = step=30
=== LABELS ===
label: rally
  attr: rally_number
  attr: start_time
  attr: end_time
  attr: shots_count
  attr: ending_reason
  attr: score_before (A-B)
  attr: score_after (A-B)
  attr: notes
```

23. assistant (2026-06-06T06:45:59.822Z)

Confirmed: `stop\_frame=21270`, `frame\_filter step=30`, `size=710`, `mode=interpolation`, label `rally` with the listed attributes (no `rotation` attribute under the label definition ? let me check it's per-image). Now the critical part: how the rally objects are encoded (tracks vs per-image boxes) and verifying the frame-range vs start_time/end_time hypothesis.

24. user (2026-06-06T06:46:03.900Z)

```
root children tags + counts:
Counter({'image': 710, 'version': 1, 'meta': 1})
num tracks = 0 num images = 710
```

25. assistant (2026-06-06T06:46:08.460Z)

It's image-based, not track-based (710 `<image>` elements). So the temporal signal must be reconstructed from per-image boxes. Let me extract per-rally frame ranges and the start_time/end_time attributes to verify the core hypothesis.

26. user (2026-06-06T06:46:18.820Z)

```
images with >=1 rally box: 281
total rally boxes: 318
distinct rally_numbers: 24
```

```
SAMPLE image attribs: {'id': '150', 'name': 'frame_000150', 'width': '1920', 'height': '1080'}
SAMPLE box attribs: {'label': 'rally', 'source': 'manual', 'occluded': '0', 'xtl': '1134.10',
'yt1': '434.73', 'xbr': '1286.29', 'ybr': '705.46', 'z_order': '0'}
SAMPLE attrs: {'rally_number': '2', 'start_time': '2', 'end_time': '2.1', 'shots_count': '2',
'ending_reason': 'forced_error', 'score_before (A-B)': None, 'score_after (A-B)': None, 'notes':
None}
```

rally	min_f	max_f	start_attr	end_attr	min/900	max/900	min/30	max/30
1	180	1260	0.2	1.4	0.200	1.400	6.00	42.00
2	150	1950	2	2.1	0.167	2.167	5.00	65.00
3	2670	3060	2.9	3.4	2.967	3.400	89.00	102.00
4	3720	4020	4.13	4.14	4.133	4.467	124.00	134.00
5	4320	4560	4.8	5.06	4.800	5.067	144.00	152.00
6	4980	5340	5.53	5.9	5.533	5.933	166.00	178.00
7	5730	6690	6.36	6.66	6.367	7.433	191.00	223.00
8	7080	7170	7.86	7.96	7.867	7.967	236.00	239.00
9	7590	7860	8.43	8.73	8.433	8.733	253.00	262.00

10	8280	8460	9.2	9.4	9.200	9.400	276.00	282.00
11	8970	9240	9.96	10.23	9.967	10.267	299.00	308.00
12	9780	9900	10.86	10.96	10.867	11.000	326.00	330.00
13	10410	10680	11.56	11.86	11.567	11.867	347.00	356.00
14	10980	11160	12.2	12.4	12.200	12.400	366.00	372.00
15	11550	11760	12.83	13.06	12.833	13.067	385.00	392.00
16	12330	13380	13.7	13.93	13.700	14.867	411.00	446.00
17	14160	14250	15.73	15.73	15.733	15.833	472.00	475.00
18	14580	15120	16.2	16.8	16.200	16.800	486.00	504.00
19	15750	15900	17.5	17.66	17.500	17.667	525.00	530.00
20	16230	16830	18.03	18.7	18.033	18.700	541.00	561.00
21	17670	17820	19.63	19.8	19.633	19.800	589.00	594.00
22	18480	18720	20.53	20.8	20.533	20.800	616.00	624.00
23	19260	19500	21.4	21.66	21.400	21.667	642.00	650.00
24	19920	20160	22.13	22.4	22.133	22.400	664.00	672.00

27. assistant (2026-06-06T06:46:34.276Z)

This is the decisive evidence. The hypothesis is **confirmed and refined**:

- `start\_time` attribute ? `min\_frame / 900` and `end\_time` attribute ? `max\_frame / 900` (the columns match almost exactly, e.g. rally 24: start=22.13?19920/900=22.133, end=22.4?20160/900=22.4).

- So the annotators (or a Roora tool) computed the time attributes by dividing the source frame number by **900**, not by the true source fps (~29.97). The real wall-clock time of the rally is `frame / fps\_source`. With `frame/30` (the sampling step times nothing useful) you'd get 664?672s which exceeds a 100s video ? wrong. With `frame / 29.97` you'd get ~664?672s too. Wait ? let me reconcile this with the on-disk video durations. The frame numbers go up to 21270 = stop_frame. If source fps?29.97, that's 21270/29.97 ? 710s ? 11.8 min, which does NOT match the 100.7s on-disk Video 1. This confirms the second hypothesis: the annotated source is a different, longer (~12 min) video, not the 100.7s on-disk MP4.

Let me verify the "frame is the true signal, 900 is wrong" point and pin down the per-rally interval gaps/overlaps, plus confirm there's no `rotation` attribute and check the COCO/VOC for an alternative frame encoding. Let me also confirm the exact fps needed.

28. user (2026-06-06T06:46:47.613Z)

ALL attribute names seen on boxes: ['end_time', 'ending_reason', 'notes', 'rally_number', 'score_after (A-B)', 'score_before (A-B)', 'shots_count', 'start_time']

distinct frame-id gaps between consecutive SAMPLED frames: [30]

min frame id 0 max frame id 21270 count 710

fps=29.97: span 6.0s -> 672.7s ; adjacent-overlaps=1

fps=30.0: span 6.0s -> 672.0s ; adjacent-overlaps=1

fps=59.94: span 3.0s -> 336.3s ; adjacent-overlaps=1

29. assistant (2026-06-06T06:46:58.433Z)

Key findings confirmed:

- No `rotation` attribute on boxes (the established "rotation" field is not present in this export; attribute set is exactly the 8 listed).
- Sampled frames are strictly step=30 (0, 30, 60, ... 21270), 710 frames.
- The frame-range span is ~6s to ~672s at ~30fps ? an ~11-minute video, confirming the annotated source is NOT the 100.7s on-disk MP4.
- There is **1 adjacent overlap** at any fps (rally min/max boxes overlap a neighbor) ? the adapter must handle de-overlap or at least be aware GT intervals can touch/overlap.

One nuance: the frame range is from sampled boxes only. With step=30, the true rally boundary is within ±30 frames (±1s) of the observed min/max box. Let me quickly check the COCO export to see if it carries any frame/time info the VOC/XML lacks (it likely only has image filenames + bboxes, no per-image rally attributes), to finalize which format is easiest. Let me also confirm the VOC has the rally attributes or not.

30. user (2026-06-06T06:47:03.276Z)

```

COCO top keys: ['licenses', 'info', 'categories', 'images', 'annotations']
num images: 710
num annotations: 318
categories: [(1, 'rally')]
image[0]: {'id': 1, 'width': 1920, 'height': 1080, 'file_name': 'frame_000000.png', 'license':
0, 'flickr_url': '', 'coco_url': '', 'date_captured': 0}
annotation[0] keys: ['id', 'image_id', 'category_id', 'segmentation', 'area', 'bbox', 'iscrowd',
'attributes']
annotation[0]: {'id': 1, 'image_id': 6, 'category_id': 1, 'segmentation': [], 'area':
41202.398700000002, 'bbox': [1134.1, 434.73, 152.19, 270.73], 'iscrowd': 0, 'attributes':
{'rally_number': 2.0, 'start_time': '2', 'end_time': '2.1', 'shots_count': 2.0, 'ending_reason':
'forced_error', 'score_before (A-B)': '', 'score_after (A-B)': '', 'notes': '', 'occluded':
False, 'rotation': 0.0}}
non-standard annotation fields: {'attributes': 318}
sample attributes field: {'rally_number': 2.0, 'start_time': '2', 'end_time': '2.1',
'shots_count': 2.0, 'ending_reason': 'forced_error', 'score_before (A-B)': '', 'score_after
(A-B)': '', 'notes': '', 'occluded': False, 'rotation': 0.0}

```

31. assistant (2026-06-06T06:47:12.483Z)

This clarifies a lot. The COCO export carries the same rally attributes in `annotations.attributes` (including `rotation`, which is CVAT's bbox rotation field, default 0.0 ? not a rally semantic). But COCO's frame number is only recoverable via `images.file\_name` (`frame\_000000.png`) joined through `image\_id`. The CVAT-video XML has the frame number directly as `image.id` / `image.name`, which is cleaner.

Let me verify one more thing ? whether the start_time/end_time attributes for any rally are reliable enough to use directly vs. the frame range, by checking the off-by-900 thesis more rigorously, and confirm the frame number embedded in COCO filenames matches the CVAT image ids. I already have enough but let me quickly confirm the COCO image_id?file_name?frame mapping reconstructs the same ranges, to definitively rank the formats.

32. user (2026-06-06T06:47:17.761Z)

COCO reconstructs 24 rallies
max |attr - frame/900| over all start/end: 1.8333 (=>attr==frame/900 confirms)
distinct rotation values: {0.0}

33. assistant (2026-06-06T06:48:00.727Z)

The max deviation of 1.83s is from a couple of rallies where the annotator rounded the time attribute coarsely (e.g. rally 2: start_attr=2 vs min_frame 150/900=0.167; that's the outlier). The pattern holds for the vast majority. The decisive point stands: **the `start\_time`/`end\_time` attributes are unreliable** (computed against a wrong 900 divisor and hand-rounded), so the **per-rally frame range is the trustworthy temporal signal**, and it must be divided by the **true source fps**, which is unknown from the export alone (must come from the source video / Roora).

`rotation` is uniformly 0.0 ? it's CVAT's bbox-rotation attribute, not a rally field. I have everything needed. Let me record findings and return the structured output.

The complete contract:

- **Expected GT format** (eval/annotations.py:1-76): a per-video CSV with optional `start,end` header, each row two timestamps in **decimal seconds** or `MM:SS`/`HH:MM:SS`; `start < end` enforced (raises otherwise); blank/`#` lines skipped. Intervals returned sorted by start. Semantics in metrics.py:12-29 ? half-open `(start,end)` durations used for temporal IoU.
- **How providers load** (file_provider.py:60-63 ? load_annotations; harness.py:47-63; regression.py:144-147; batch.py reads the collector manifest's per-video CSV).
- **Collector exporter** (curate.py:422-518, db.py:509-528) emits exactly `start,end` decimal-seconds CSVs + manifest.json ? this is the canonical target the adapter should match.

34. user (2026-06-06T06:48:24.022Z)

Structured output provided successfully

35. user (2026-06-06T06:48:28.646Z)

[Request interrupted by user]